

# 4주차 예습과제\_김유진

## [4장] 분류

- GBM
- XGBoost
- LightGBM
- 랜덤 포레스트

## 05 GBM(Gradient Boosting Machine)

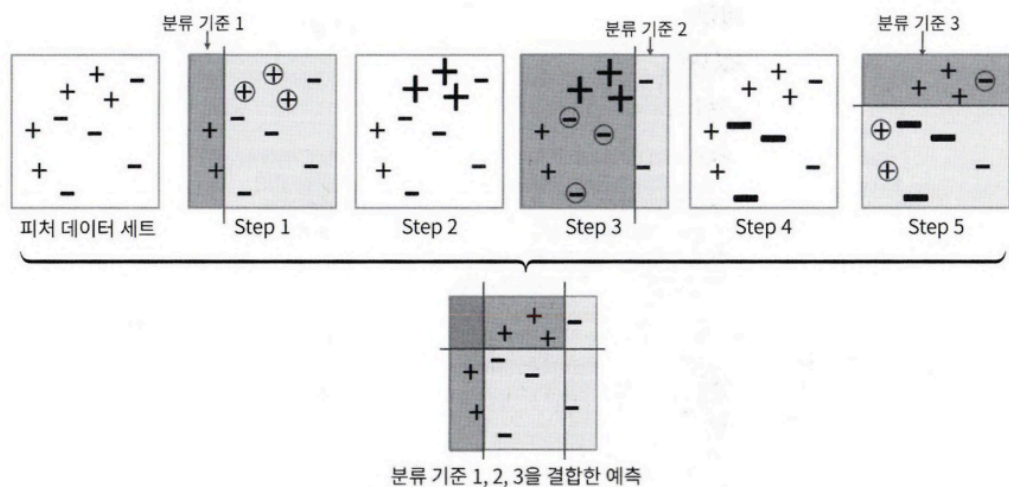
### GBM의 개요 및 실습

**부스팅 알고리즘:** 여러 개의 약한 학습기를 순차적으로 학습-예측하면서 잘못 예측한 데이터에 가중치 부여를 통해 오류를 개선해 나가면서 학습하는 방식

#### 1. AdaBoost

개별 약한 학습기는 각각 가중치를 부여해 결합함

예를 들어, 첫 번째 학습기에 가중치 0.3, 두 번째 학습기에 가중치 0.5, 세 번째 학습기에 가중치 0.8을 부여한 후 모두 결합해 예측을 수행



## 2. GBM

- AdaBoost와 비슷하지만 가중치 업데이트를 경사 하강법으로 이용한다는 차이점 있음
- GBM은 랜덤 포레스트보다 예측 성능이 조금 뛰어난 경우가 많지만 수행 시간이 오래 걸림

**경사하강법:** 반복 수행을 통해 오류를 최소화할 수 있도록 가중치의 업데이트 값을 도출하는 기법

## GBM 하이퍼 파라미터 소개

|                      |                                                                          |                   |
|----------------------|--------------------------------------------------------------------------|-------------------|
| <b>loss</b>          | 경사하강법에서 사용할 비용 함수 지정                                                     | default: deviance |
| <b>learning_rate</b> | GBM이 학습을 진행할 때마다 적용하는 학습률.<br>Weak learner가 순차적으로 오류 값을 보정해나가는 데 적용하는 계수 | default: 0.1      |
| <b>n_estimators</b>  | weak learner의 개수. 개수가 많아질수록 예측 성능은 일정 수준까지 좋아지지만 시간 오래 걸림.               | default: 100      |
| <b>subsample</b>     | weak learner가 학습에 사용하는 데이터의 샘플링 비율                                       | default: 1        |

# 06 XGBoost

## XGBoost 개요

| 항목                      | 설명                                                                                                                                                                                                                       |
|-------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 뛰어난 예측 성능               | 일반적으로 분류와 회귀 영역에서 뛰어난 예측 성능을 발휘합니다.                                                                                                                                                                                      |
| GBM 대비 빠른 수행 시간         | 일반적인 GBM은 순차적으로 Weak learner가 가중치를 증감하는 방법으로 학습하기 때문에 전반적으로 속도가 느립니다. 하지만 XGBoost는 병렬 수행 및 다양한 기능으로 GBM에 비해 빠른 수행 성능을 보장합니다. 아쉽게도 XGBoost가 일반적인 GBM에 비해 수행 시간이 빠르다는 것이지, 다른 머신러닝 알고리즘(예를 들어 랜덤 포레스트)에 비해서 빠르다는 의미는 아닙니다. |
| 과적합 규제 (Regularization) | 표준 GBM의 경우 과적합 규제 기능이 없으나 XGBoost는 자체에 과적합 규제 기능으로 과적합에 좀 더 강한 내구성을 가질 수 있습니다.                                                                                                                                           |
| Tree pruning (나무 가지치기)  | 일반적으로 GBM은 분할 시 부정 손실이 발생하면 분할을 더 이상 수행하지 않지만, 이러한 방식도 자칫 지나치게 많은 분할을 발생시킬 수 있습니다.<br>다른 GBM과 마찬가지로 XGBoost도 max_depth 파라미터로 분할 깊이를 조정하기도 하지만, tree pruning으로 더 이상 긍정 이익이 없는 분할을 가지치기 해서 분할 수를 더 줄이는 추가적인 장점을 가지고 있습니다.  |
| 자체 내장된 교차 검증            | XGBoost는 반복 수행 시마다 내부적으로 학습 데이터 세트와 평가 데이터 세트에 대한 교차 검증을 수행해 최적화된 반복 수행 횟수를 가질 수 있습니다.<br>지정된 반복 횟수가 아니라 교차 검증을 통해 평가 데이터 세트의 평가 값이 최적화 되면 반복을 중간에 멈출 수 있는 조기 중단 기능이 있습니다.                                               |
| 결손값 자체 처리               | XGBoost는 결손값을 자체 처리할 수 있는 기능을 가지고 있습니다.                                                                                                                                                                                  |

- **일반 파라미터:** 일반적으로 실행 시 스레드의 개수나 silent 모드 등의 선택을 위한 파라미터로서 디폴트 파라미터 값을 바꾸는 경우는 거의 없음

#### 주요 일반 파라미터

- **booster:** gbtree(tree based model) 또는 gblinear( linear model) 선택. 디폴트는 gbtree입니다.
- **silent:** 디폴트는 0이며, 출력 메시지를 나타내고 싶지 않을 경우 1로 설정합니다.
- **nthread:** CPU의 실행 스레드 개수를 조정하며, 디폴트는 CPU의 전체 스레드를 다 사용하는 것입니다. 멀티 코어/스레드 CPU 시스템에서 전체 CPU를 사용하지 않고 일부 CPU만 사용해 ML 애플리케이션을 구동하는 경우에 변경합니다.

- **부스터 파라미터:** 트리 최적화, 부스팅, regularization 등과 같은 파라미터 등을 지칭함

## 주요 부스터 파라미터

- `eta [default=0.3, alias: learning_rate]`: GBM의 학습률(learning rate)과 같은 파라미터입니다. 0에서 1 사이의 값을 지정하며 부스팅 스텝을 반복적으로 수행할 때 업데이트되는 학습률 값. 파이썬 래퍼 기반의 xgboost를 이용할 경우 디폴트는 0.3, 사이킷런 래퍼 클래스를 이용할 경우 `eta`는 `learning_rate` 파라미터로 대체되며, 디폴트는 0.1입니다. 보통은 0.01 ~ 0.2 사이의 값을 선호합니다.
- `num_boost_rounds`: GBM의 `n_estimators`와 같은 파라미터입니다.
- `min_child_weight [default=1]`: 트리에서 추가적으로 가지를 나눌지를 결정하기 위해 필요한 데이터들의 `weight` 총합. `min_child_weight`이 클수록 분할을 자제합니다. 과적합을 조절하기 위해 사용됩니다.
- `gamma [default=0, alias: min_split_loss]`: 트리의 리프 노드를 추가적으로 나눌지를 결정할 최소 손실 감소 값입니다. 해당 값보다 큰 손실(loss)이 감소된 경우에 리프 노드를 분리합니다. 값이 클수록 과적합 감소 효과가 있습니다.
- `max_depth [default=6]`: 트리 기반 알고리즘의 `max_depth`와 같습니다. 0을 지정하면 깊이에 제한이 없습니다. `Max_depth`가 높으면 특정 피쳐 조건에 특화되어 룰 조건이 만들어지므로 과적합 가능성이 높아지며 보통은 3~10 사이의 값을 적용합니다.
- `sub_sample [default=1]`: GBM의 `subsample`과 동일합니다. 트리가 커져서 과적합되는 것을 제어하기 위해 데이터를 샘플링하는 비율을 지정합니다. `sub_sample=0.5`로 지정하면 전체 데이터의 절반을 트리를 생성하는 데 사용합니다. 0에서 1 사이의 값이 가능하나 일반적으로 0.5 ~ 1 사이의 값을 사용합니다.
- `colsample_bytree [default=1]`: GBM의 `max_features`와 유사합니다. 트리 생성에 필요한 피쳐(칼럼)를 임의로 샘플링하는 데 사용됩니다. 매우 많은 피쳐가 있는 경우 과적합을 조정하는 데 적용합니다.
- `lambda [default=1, alias: reg_lambda]`: L2 Regularization 적용 값입니다. 피쳐 개수가 많을 경우 적용을 검토하며 값이 클수록 과적합 감소 효과가 있습니다.
- `alpha [default=0, alias: reg_alpha]`: L1 Regularization 적용 값입니다. 피쳐 개수가 많을 경우 적용을 검토하며 값이 클수록 과적합 감소 효과가 있습니다.

- **학습 태스크 파라미터**: 학습 수행 시의 객체 함수, 평가를 위한 지표 등을 설정하는 파라미터

## 학습 태스크 파라미터

- **objective**: 최솟값을 가져야 할 손실 함수를 정의합니다. XGBoost는 많은 유형의 손실함수를 사용할 수 있습니다. 주로 사용되는 손실함수는 이진 분류인지 다중 분류인지에 따라 달라집니다.
- **binary:logistic**: 이진 분류일 때 적용합니다.
- **multi:softmax**: 다중 분류일 때 적용합니다. 손실함수가 multi:softmax일 경우에는 레이블 클래스의 개수인 num\_class 파라미터를 지정해야 합니다.
- **multi:softprob**: multi:softmax와 유사하나 개별 레이블 클래스의 해당되는 예측 확률을 반환합니다.
- **eval\_metric**: 검증에 사용되는 함수를 정의합니다. 기본값은 회귀인 경우는 rmse, 분류일 경우에는 error입니다. 다음은 eval\_metric의 값 유형입니다.
  - **rmse**: Root Mean Square Error
  - **mae**: Mean Absolute Error
  - **logloss**: Negative log-likelihood
  - **error**: Binary classification error rate (0.5 threshold)
  - **merror**: Multiclass classification error rate
  - **mlogloss**: Multiclass logloss
  - **auc**: Area under the curve

- 과적합 문제 해결방법

- max\_depth 값을 낮춘다
- min\_child\_weight 값을 높인다
- gamma 값을 높인다
- subsample과 colsample\_bytree를 조종한다
- eta 값을 낮춘다

## 사이킷런 래퍼 XGBoost의 개요 및 적용

- eta → learning\_rate
- sub\_sample → subsample
- lambda → reg\_lambda
- alpha → reg\_alpha

## 07 LightGBM

- 장점: XGBoost보다 학습에 걸리는 시간이 훨씬 적다.  
메모리 사용량도 상대적으로 적다  
카테고리형 피처의 자동 변환과 최적 분할
- 단점: 적은 데이터 세트에 적용할 경우 과적합이 발생하기 쉽다

## LightGBM 하이퍼 파라미터

- `max_depth` [default=-1]: 트리 기반 알고리즘의 `max_depth`와 같습니다. 0보다 작은 값을 지정하면 깊이에 제한이 없습니다. 지금까지 소개한 Depth wise 방식의 트리와 다르게 LightGBM은 Leaf wise 기반이므로 깊이가 상대적으로 더 깊습니다.
- `min_data_in_leaf` [ default = 20]: 결정 트리의 `min_samples_leaf`와 같은 파라미터입니다. 하지만 사이킷런 래퍼 LightGBM 클래스인 `LightGBMClassifier`에서는 `min_child_samples` 파라미터로 이름이 변경됩니다. 최종 결정 클래스인 리프 노드가 되기 위해서 최소한으로 필요한 레코드 수이며, 과적합을 제어하기 위한 파라미터입니다.
- `num_leaves` [ default = 31 ]: 하나의 트리가 가질 수 있는 최대 리프 개수입니다.
- `boosting` [ default = gbdt ]: 부스팅의 트리를 생성하는 알고리즘을 기술합니다.
  - `gbdt`: 일반적인 그래디언트 부스팅 결정 트리
  - `rf`: 랜덤 포레스트
- `bagging_fraction` [ default = 1.0 ]: 트리가 커져서 과적합되는 것을 제어하기 위해서 데이터를 샘플링하는 비율을 지정합니다. 사이킷런의 GBM과 `XGBClassifier`의 `subsample` 파라미터와 동일하기에 사이킷런 래퍼 LightGBM인 `LightGBMClassifier`에서는 `subsample`로 동일하게 파라미터 이름이 변경됩니다.
- `feature_fraction` [ default = 1.0 ]: 개별 트리를 학습할 때마다 무작위로 선택하는 피처의 비율입니다. 과적합을 막기 위해 사용됩니다. GBM의 `max_features`와 유사하며, `XGBClassifier`의 `colsample_bytree`와 똑같므로 LightGBM Classifier에서는 동일하게 `colsample_bytree`로 변경됩니다.
- `lambda_l2` [default=0.0 ]: L2 regulation 제어를 위한 값입니다. 피처 개수가 많을 경우 적용을 검토하며 값이 클수록 과적합 감소 효과가 있습니다. `XGBClassifier`의 `reg_lambda`와 동일하므로 LightGBMClassifier에서는 `reg_lambda`로 변경됩니다.
- `lambda_l1` [ default = 0.0 ]: L1 regulation 제어를 위한 값입니다. L2와 마찬가지로 과적합 제어를 위한 것이며, `XGBClassifier`의 `reg_alpha`와 동일하므로 LightGBMClassifier에서는 `reg_alpha`로 변경됩니다.

## 하이퍼 파라미터 튜닝 방안

기본 튜닝 방안: `num_leaves`의 개수를 중심으로

`min_child_samples`(`min_data_in_leaf`), `max_depth` 를 함께 조정하면서 모델의 복잡도를 줄이는 것

- **num\_leaves**: 개별 트리가 가질 수 있는 최대 리프의 개수이고 LightGBM 모델의 복잡도를 제어하는 주요 파라미터. 개수 ↑ → 정확도 ↑, 복잡도 ↑, 과적합 영향 ↑
- **min\_data\_in\_leaf**: (=min\_child\_samples) 과적합 개선하기 위한 중요 파라미터. 큰 값으로 설정하면 트리가 깊어지는 것 방지함
- **max\_depth**: 깊이의 크기 제한

## 파이썬 래퍼 LightGBM과 사이킷런 래퍼 XGBoost, LightGBM 하이퍼 파라미터 비교

| 유형    | 파이썬 래퍼 LightGBM         | 사이킷런 래퍼 LightGBM      | 사이킷런 래퍼 XGBoost       |
|-------|-------------------------|-----------------------|-----------------------|
| 파라미터명 | num_iterations          | n_estimators          | n_estimators          |
|       | learning_rate           | learning_rate         | learning_rate         |
|       | max_depth               | max_depth             | max_depth             |
|       | min_data_in_leaf        | min_child_samples     | N/A                   |
|       | bagging_fraction        | subsample             | subsample             |
|       | feature_fraction        | colsample_bytree      | colsample_bytree      |
|       | lambda_l2               | reg_lambda            | reg_lambda            |
|       | lambda_l1               | reg_alpha             | reg_alpha             |
|       | early_stopping_round    | early_stopping_rounds | early_stopping_rounds |
|       | num_leaves              | num_leaves            | N/A                   |
|       | min_sum_hessian_in_leaf | min_child_weight      | min_child_weight      |

## 08 베이지안 최적화 기반의 HyperOpt를 이용한 하이퍼 파라미터 튜닝

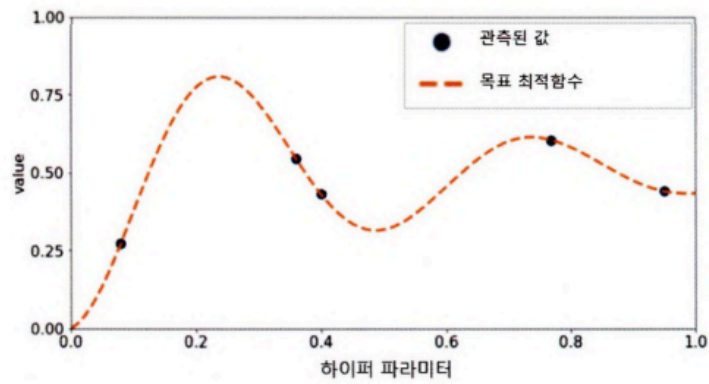
### 베이지안 최적화 개요

**베이지안 최적화**: 목적 함수 식을 제대로 알 수 없는 **블랙 박스 형태의 함수**에서 최대 또는 최소 함수 반환 값

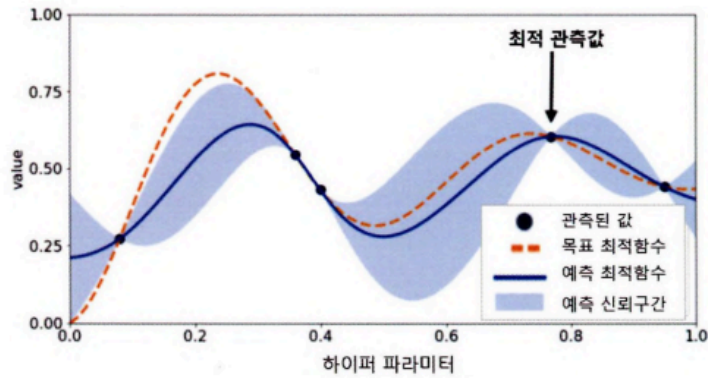
을 만드는 최적 입력값을 가능한 적은 시도를 통해 빠르고 효과적으로 찾아주는 방식

- 베이지안 확률을 기반으로 함.
- **\*베이지안 확률**: 새로운 사건의 관측이나 새로운 샘플 데이터를 기반으로 사후 확률을 개선해 나가는 것

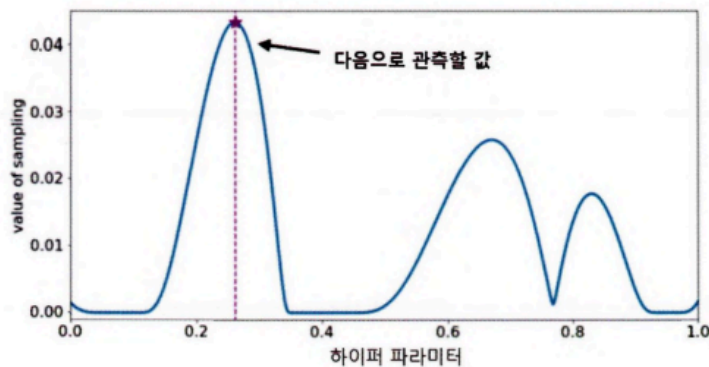
- **대체 모델:** 획득 함수로부터 최적 함수를 예측할 수 있는 입력값을 추천 받은 뒤 이를 기반으로 최적 함수 모델을 개선
- **획득 함수:** 개선된 대체 모델을 기반으로 최적 입력값을 계산



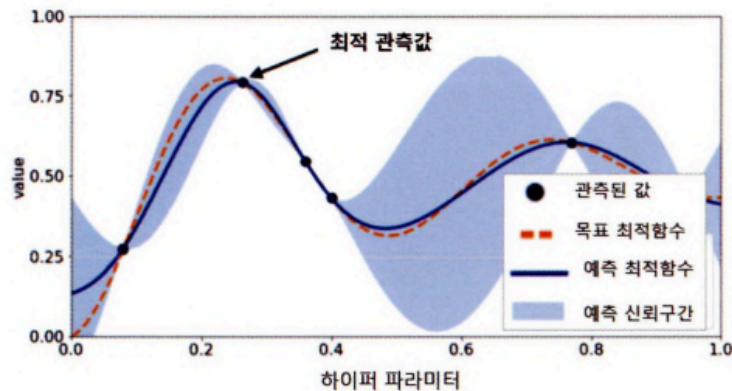
- **Step 2:** 관측된 값을 기반으로 대체 모델은 최적 함수를 추정합니다. 아래 그림에서 파란색 실선은 대체 모델이 추정한 최적 함수입니다. 옅은 파란색으로 되어 있는 영역은 예측된 함수의 신뢰 구간입니다. 추정된 함수의 결과값 오류 편차를 의미하며 추정 함수의 불확실성을 나타냅니다. 최적 관측값은 y축 value에서 가장 높은 값을 가질 때의 하이퍼 파라미터입니다.



- **Step 3:** 추정된 최적 함수를 기반으로 획득 함수(Acquisition Function)는 다음으로 관측할 하이퍼 파라미터 값을 계산합니다. 획득 함수는 이전의 최적 관측값보다 더 큰 최댓값을 가질 가능성이 높은 지점을 찾아서 다음에 관측할 하이퍼 파라미터를 대체 모델에 전달합니다.



- **Step 4:** 획득 함수로부터 전달된 하이퍼 파라미터를 수행하여 관측된 값을 기반으로 대체 모델은 갱신되어 다시 최적 함수를 예측 추정합니다.



## HyperOpt 사용하기

### 로직 구성

1. 입력 변수명과 입력값의 검색 공간 설정
2. 목적 함수 설정
3. 목적 함수의 반환 최솟값을 가지는 최적 입력값 유추

|                                  |                                                                                                  |
|----------------------------------|--------------------------------------------------------------------------------------------------|
| hp.quniform(label, low, high, q) | label로 지정된 입력값 변수 검색 공간을 최솟값 low에서 최댓값 high까지 q의 간격을 가지고 설정                                      |
| hp.uniform(low, label, high)     | 최솟값 low에서 최댓값 high까지 정규 분포 형태의 검색 공간 설정                                                          |
| hp.radicent(label, upper)        | 0부터 최댓값 upper까지 random한 정숫값으로 검색 공간 설정                                                           |
| hp.loguniform(label, low, high)  | $\exp(\text{uniform}(\text{low}, \text{high}))$ 값을 반환하며, 반환 값의 log 변환된 값은 정규 분포 형태를 가지는 검색 공간 설정 |
| hp.choice(label, options)        | 검색 값이 문자열 또는 문자열과 숫자값이 섞여 있을 경우 설정                                                               |

- fn: objective\_func와 같은 목적 함수
- space: search\_space와 같은 검색 공간 딕셔너리
- algo: 베이지안 최적화 적용 알고리즘
- max\_evals: 최적 입력값을 찾기 위한 입력값 시도 횟수
- trials: 최적 입력값을 찾기 위해 시도한 입력값 및 해당 입력값의 목적 함수 반환값 결과를 저장하는 데 사용
- rstate: fmin()을 수행할 때마다 동일한 결괏값을 가질 수 있도록 설정하는 랜덤 시드

## 10 분류 실습 - 캐글 신용카드 사기 검출

### 언더 샘플링과 오버 샘플링의 이해

- **언더 샘플링**: 많은 데이터 세트를 적은 데이터 세트 수준으로 감소시키는 방식.
- **오버 샘플링**: 이상 데이터와 같은 적은 데이터 세트를 증식하여 학습을 위한 충분한 데이터 확보하는 방법

### 이상치 데이터 제거 후 모델 학습/예측/평가

- **이상치 데이터**: 전체 데이터의 패턴에서 벗어난 이상 값을 가진 데이터
- **사분위**: 전체 데이터를 값이 높은 순으로 정렬하고, 이를 1/4씩으로 구간을 분할하는 것을 지칭함

## 11 스택킹 앙상블

- **스택킹**: 개별적인 여러 알고리즘을 서로 결합해 예측 결과를 도출
- **개별 알고리즘으로 예측한 데이터를 기반으로 다시 예측을 수행한다는 차이점**
- 개별 알고리즘의 예측 결과 데이터 세트를 최종적인 메타 데이터 세트로 만들어 별도의 ML 알고리즘으로 최종학습을 수행하고 테스트 데이터 기반으로 다시 최종 예측 수행하는 방식

